



Strathmore University

@iLabAfrica Centre

Session 5: Unsupervised Learning & Dimensionality Reduction

Discovering structure in unlabelled data

Javier Serrano

Applied Machine Learning

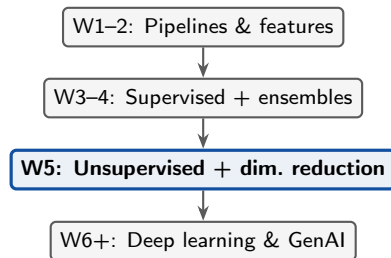
Master in Data Science & Analytics

Where we are in the course

Part I closes: from labelled prediction to structure discovery

- **Weeks 1–2:** the end-to-end lifecycle; pipelines, leakage-safe feature engineering.
- **Weeks 3–4:** supervised learning done rigorously; ensembles and tuning.
- **Today:** we drop the labels. What can we learn from X alone?
- **Next (Week 6):** deep learning fundamentals, where learned representations replace hand-built ones.

Looking ahead: autoencoders (Week 6) and LLM embeddings (Week 10) are dimensionality reduction by another name.



Session objectives

Intended learning outcomes for Week 5

By the end of this session you will be able to:

1. Formulate clustering, density estimation and dimensionality reduction as optimisation problems over unlabelled data.
2. Apply and diagnose **K-Means**, **DBSCAN** and **Gaussian Mixture Models**; select k with inertia, silhouette and information criteria.
3. Derive and apply **PCA** via variance maximisation / SVD; choose the target dimension from the explained-variance spectrum.
4. Build an unsupervised **customer-segmentation and anomaly-detection** workflow for thin-file mobile-money users.

Running thread: millions of mobile-money users transact daily; almost none carry a “defaulted / repaid” label yet. Structure must be found *before* it can be supervised.

Why Unsupervised Learning

Clustering with K-Means

Density-Based Clustering: DBSCAN

Gaussian Mixture Models

The Curse of Dimensionality & PCA

Lab, Reading & Wrap-Up

Most data is unlabelled

The economics of labels

- Supervised learning needs (x_i, y_i) pairs; labels are **expensive, slow and biased** by whoever assigns them.
- A lender entering a new market has **transaction histories for millions** of wallets but repayment outcomes for only the few thousand it has already lent to.
- Unsupervised learning works with $\mathcal{D} = \{x_1, \dots, x_m\}$ alone: it finds *structure*, groups, densities, low-dimensional coordinates.
- LeCun's cake: unsupervised learning is the cake, supervised the icing, RL the cherry.

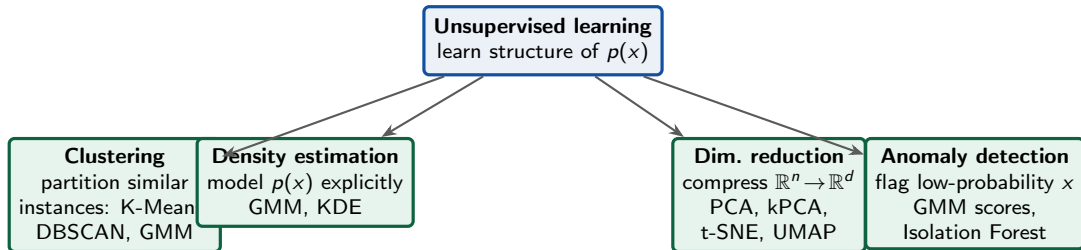
A typical fintech data estate



↓
clustering, density estimation,
dimensionality reduction, anomaly detection

The unsupervised task taxonomy

Four problems, one theme: structure without labels



- These are not silos: a fitted GMM *is* a density model, a clusterer *and* an anomaly detector.
- In production, unsupervised outputs usually **feed a supervised or business process**: segments $\rightarrow$ pricing tiers; anomaly scores $\rightarrow$ fraud queues; PCA coordinates $\rightarrow$ model features.

Week 5 through the credit-scoring thread

Where unsupervised learning enters the lending pipeline



- **Segmentation:** gig workers, market traders, salaried users and agri-cycle earners have different cash-flow rhythms, one global scorecard underserves all of them.
- **Anomaly detection:** SIM-swap fraud and agent collusion appear as low-density behaviour long before any chargeback label exists.
- **Compression:** 200 correlated behavioural features → a compact representation that trains faster and generalises better (Week 4 models benefit directly).

Lab, Reading & Wrap-Up

K-Means: the objective

Minimising within-cluster variance (inertia)

Partition $\{x_1, \dots, x_m\}$ into k clusters with centroids $\mu_1, \dots, \mu_k$ by minimising the inertia:

$$J = \sum_{i=1}^m \min_{j \in \{1, \dots, k\}} \|x_i - \mu_j\|^2$$

- NP-hard in general; **Lloyd's algorithm** finds a local optimum by alternating two steps.
- Equivalent to a *hard*, spherical, equal-variance special case of the GMM we meet later today.
- Complexity per iteration: $\mathcal{O}(mkn)$, fast and scalable.

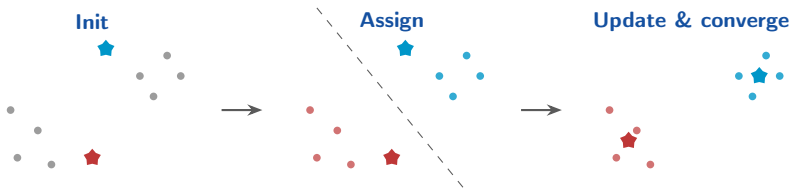
Lloyd's algorithm

1. Initialise centroids (use k-means++ seeding).
2. **Assign:** $c_i \leftarrow \arg \min_j \|x_i - \mu_j\|^2$
3. **Update:** $\mu_j \leftarrow \frac{1}{|C_j|} \sum_{i \in C_j} x_i$
4. Repeat 2–3 until assignments stop changing.

J decreases monotonically $\Rightarrow$ guaranteed convergence, but only to a **local** minimum, hence `n_init` restarts.

K-Means: geometry of convergence

Assign–update alternation



- Decision boundaries between centroids are **linear** (a Voronoi tessellation), K-Means can only carve convex, roughly spherical regions.
- **Always standardise features first:** a feature in KES (thousands) will dominate one in counts (tens) inside the Euclidean norm.

K-Means in scikit-learn

Segmenting mobile-money wallets

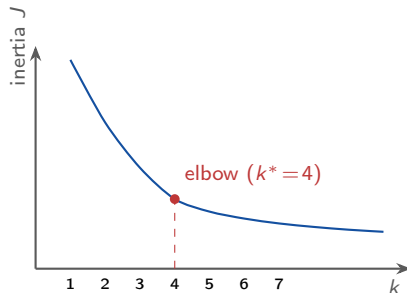
```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
# X: one row per wallet -- 90-day behavioural aggregates
seg = Pipeline([
    ("scale", StandardScaler()),          # NEVER skip before K-Means
    ("km", KMeans(n_clusters=5, init="k-means++",
                  n_init=10, random_state=42)),
])
labels = seg.fit_predict(X)
km = seg.named_steps["km"]
print(km.inertia_)                      # J at convergence
# centroids back in original units for business profiling:
profile = seg.named_steps["scale"].inverse_transform(km.cluster_centers_)
```

- `n_init=10`: ten restarts, keep the lowest inertia, cheap insurance against local minima. For $m \gtrsim 10^6$ wallets use `MiniBatchKMeans`.

Choosing k I: the elbow heuristic

Inertia always decreases, look for diminishing returns

- Inertia $J(k)$ is **monotonically decreasing** in k (with $k = m$, $J = 0$), so we cannot just minimise it.
- Plot $J(k)$ and find the **elbow**: the point where adding a cluster stops buying much variance reduction.
- Coarse but fast; use it to bracket a candidate range, then refine with the silhouette.
- k must also be **actionable**: five segments can be priced and marketed to; forty cannot.



Choosing k II: the silhouette score

A per-instance measure of cluster fit

With $a(i)$ = mean distance to own cluster, $b(i)$
= mean distance to *nearest other* cluster:

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}} \in [-1, 1]$$

- $s(i) \approx +1$: deep inside its cluster; ≈ 0 : boundary; < 0 : mis-assigned. **Score**
 $= \frac{1}{m} \sum_i s(i)$; pick k near the peak.
- Silhouette *diagrams* (per-cluster sorted $s(i)$) reveal weak clusters. Cost $\mathcal{O}(m^2)$: sample for large m .

```
from sklearn.metrics import  
    silhouette_score  
for k in range(2, 11):  
    km = KMeans(n_clusters=k,  
                n_init=10).fit(Xs)  
    s = silhouette_score(  
        Xs, km.labels_,  
        sample_size=20_000)  
    print(k, round(s, 3))
```

Limits of K-Means

When Voronoi geometry betrays you

- Assumes clusters are **convex**, **isotropic and similar in size**, elongated, nested or varying-density clusters get chopped.
- Every point is assigned somewhere: **no notion of noise or outliers**.
- k must be fixed in advance.
- Sensitive to scaling and initialisation (mitigated by `k-means++` + restarts).

These failures motivate **DBSCAN** (density, noise, shape) and **GMM** (elliptical, soft, probabilistic).



Roadmap

Why Unsupervised Learning

Clustering with K-Means

Density-Based Clustering: DBSCAN

Gaussian Mixture Models

The Curse of Dimensionality & PCA

Applied Thread: Segmenting Thin-File Borrowers

Lab, Reading & Wrap-Up

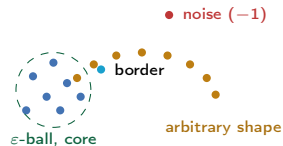
DBSCAN: clusters as dense regions

Two hyperparameters, no k

Definitions (given radius ε and `min_samples`):

- **Core point:** $\geq \text{min_samples}$ neighbours within distance ε .
- **Border point:** within ε of a core point, but not core itself.
- **Noise:** neither, labelled -1 , deliberately unassigned.

A cluster is a maximal set of density-connected core points plus their borders.



- Finds **arbitrary-shaped** clusters; number of clusters emerges from the data.
- Noise label = built-in **anomaly detector**.

DBSCAN in practice

Fraud-shaped clusters in transaction space

```
from sklearn.cluster import DBSCAN
from sklearn.preprocessing import StandardScaler

Xs = StandardScaler().fit_transform(X)
db = DBSCAN(eps=0.7, min_samples=15, n_jobs=-1).fit(Xs)

labels = db.labels_                # -1 == noise / candidate anomaly
n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
noise_rate = (labels == -1).mean()
print(n_clusters, f"{noise_rate:.2%} flagged as noise")
```

- **Choosing ε :** plot the sorted distance to the k -th nearest neighbour ($k = \text{min_samples}$) and take the knee.
- **No predict():** in production, train a lightweight classifier on its labels to score new points.
- Struggles when cluster densities differ widely (a single ε cannot fit all); see HDBSCAN for a hierarchical fix.

K-Means vs DBSCAN

Choosing by data geometry and product need

	K-Means	DBSCAN
Cluster shape	convex, spherical (Voronoi)	arbitrary (density-connected)
Number of clusters	fixed in advance (k)	emerges from ϵ , <code>min_samples</code>
Outliers	none, everything assigned	explicit noise label (-1)
Scalability	excellent; MiniBatch variant	good with spatial indexes; memory-heavy at extremes
New points	<code>predict()</code> via nearest centroid	no native <code>predict()</code>
Typical fintech use	segmentation for pricing/marketing	fraud-ring and anomaly discovery

Looking ahead: Gaussian mixtures next: keep K-Means' centroid view, but make it probabilistic and elliptical.

Roadmap

Why Unsupervised Learning

Clustering with K-Means

Density-Based Clustering: DBSCAN

Gaussian Mixture Models

The Curse of Dimensionality & PCA

Applied Thread: Segmenting Thin-File Borrowers

Lab, Reading & Wrap-Up

GMM: a generative view of clustering

Clusters as components of a mixture density

Model the data as drawn from a weighted sum of k Gaussians:

$$p(x) = \sum_{j=1}^k \pi_j \mathcal{N}(x | \mu_j, \Sigma_j), \quad \pi_j \geq 0, \quad \sum_{j=1}^k \pi_j = 1$$

- Latent variable $z_i \in \{1, \dots, k\}$: which component generated x_i . Clustering = inferring z_i .
- **Soft assignments** (responsibilities), by Bayes' rule:

$$\gamma_{ij} = p(z_i = j | x_i) = \frac{\pi_j \mathcal{N}(x_i | \mu_j, \Sigma_j)}{\sum_{l=1}^k \pi_l \mathcal{N}(x_i | \mu_l, \Sigma_l)}$$

- Full covariances $\Sigma_j \Rightarrow$ **elliptical** clusters of different sizes and orientations, exactly where K-Means fails.
- K-Means is the limit of a GMM with $\Sigma_j = \sigma^2 I$, equal π_j , and hard assignments.

Fitting a GMM: Expectation–Maximisation

Maximum likelihood with latent variables

Maximise $\log p(X) = \sum_{i=1}^m \log \sum_{j=1}^k \pi_j \mathcal{N}(x_i \mid \mu_j, \Sigma_j)$, no closed form, so alternate:

E-step (soft assign): compute responsibilities γ_{ij} with current parameters.

M-step (re-estimate), with $m_j = \sum_i \gamma_{ij}$:

$$\mu_j = \frac{1}{m_j} \sum_i \gamma_{ij} x_i, \quad \pi_j = \frac{m_j}{m}$$

$$\Sigma_j = \frac{1}{m_j} \sum_i \gamma_{ij} (x_i - \mu_j)(x_i - \mu_j)^\top$$

- Log-likelihood increases every iteration $\Rightarrow$ convergence to a local optimum (same caveat as Lloyd's: restart!).
- EM *is* Lloyd's algorithm with soft, covariance-aware assignments.
- Constrain Σ_j when n is large: `covariance_type` $\in$ {full, tied, diag, spherical} trades flexibility for parameter count $\mathcal{O}(kn^2) \rightarrow \mathcal{O}(kn)$.

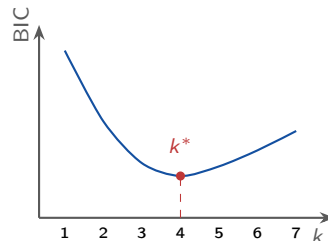
Selecting k for a GMM: information criteria

Likelihood, penalised for complexity

With p free parameters and maximised likelihood $\hat{L}$:

$$\text{BIC} = p \ln m - 2 \ln \hat{L}, \quad \text{AIC} = 2p - 2 \ln \hat{L}$$

- Fit GMMs for a range of k (and covariance types); **choose the minimum BIC/AIC**.
- BIC penalises harder ($\ln m > 2$ for $m > 7$)
 $\Rightarrow$ prefers simpler models; AIC favours predictive fit.
- Unlike inertia, these *can* be minimised directly, a principled replacement for the elbow.



BIC has a true minimum, no squinting for elbows.

GMM for anomaly detection

Low-density transactions → the fraud queue

```
from sklearn.mixture import GaussianMixture
import numpy as np

gm = GaussianMixture(n_components=5, covariance_type="full",
                     n_init=5, random_state=42).fit(Xs)

log_dens = gm.score_samples(Xs_new)    # log p(x) per instance

# Flag the least-likely 1% of transactions for manual review
threshold = np.percentile(log_dens, 1)
suspicious = Xs_new[log_dens < threshold]
resp = gm.predict_proba(Xs_new)    # soft memberships (m, k)
```

- The threshold is a **business dial**: it fixes review-queue volume, trading analyst workload against fraud caught (Week 3's cost-sensitive framing).
- Densities hold only as well as the Gaussian assumption; validate against *confirmed* fraud as labels arrive, then graduate to supervised detection.

Roadmap

Why Unsupervised Learning

Clustering with K-Means

Density-Based Clustering: DBSCAN

Gaussian Mixture Models

The Curse of Dimensionality & PCA

Lab, Reading & Wrap-Up

The curse of dimensionality

Why 200 behavioural features are a liability

As dimension n grows, geometry turns hostile:

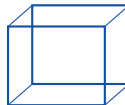
- **Sparsity:** to keep constant sample density, the required m grows *exponentially* in n .
- **Distances concentrate:** for i.i.d. dimensions, $\frac{\max_i d_i - \min_i d_i}{\min_i d_i} \rightarrow 0$, “nearest” neighbours stop being meaningfully near, degrading K-Means, DBSCAN and k-NN alike.
- Redundancy is the escape hatch: real features are **correlated**, so data concentrates near a low-dimensional structure we can recover.



$n=1$: 4 pts cover it



$n=2$: need 4^2



$n=3$: need $4^3 \Rightarrow$ curse

Consequence: before clustering 200-dimensional wallet features, reduce dimensionality, both for statistical sanity and for compute.

Two routes down: projection vs manifold learning

Flat subspaces vs curved sheets

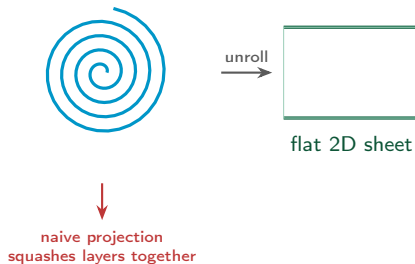
Projection (PCA):

- Data lies near a flat d -dimensional **subspace**; project onto it.
- Linear, fast, invertible (approximately), interpretable loadings.

Manifold learning (kPCA, LLE, t-SNE, UMAP):

- Data lies near a **curved** low-dimensional manifold (the Swiss roll); projection would squash distinct layers together, unroll instead.
- Nonlinear, costlier, usually not invertible.

Swiss roll manifold



PCA I: the objective

Maximise retained variance $\equiv$ minimise reconstruction error

Centre the data ($X \leftarrow X - \bar{x}$). Find unit vector w_1 capturing maximal variance:

$$w_1 = \arg \max_{\|w\|=1} \frac{1}{m} \|Xw\|^2 = \arg \max_{\|w\|=1} w^\top S w, \quad S = \frac{1}{m} X^\top X$$

- Solution: w_1 is the **top eigenvector** of the covariance matrix S ; subsequent components are the next eigenvectors, mutually orthogonal.
- Equivalent view: the same W_d **minimises squared reconstruction error** $\frac{1}{m} \sum_i \|x_i - W_d W_d^\top x_i\|^2$, variance kept = error avoided.
- In practice, computed via the **SVD** $X = U\Sigma V^\top$: principal components are the columns of V ; the projection is

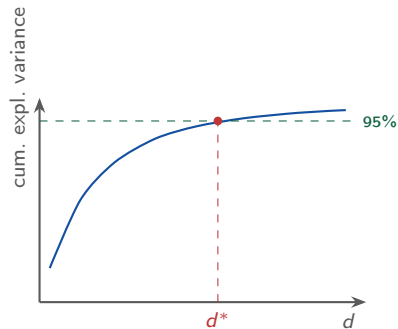
$$X_{d\text{-proj}} = XW_d, \quad W_d = V_{[:,1:d]}$$

- Eigenvalue λ_j = variance along component $j \Rightarrow$ the **explained variance ratio** $\lambda_j / \sum_l \lambda_l$.

PCA II: choosing the target dimension

Read it off the variance spectrum

- Plot cumulative explained variance vs d ; keep the smallest d reaching a threshold (95% is the usual default for preprocessing).
- For **visualisation**, $d = 2$ or 3 regardless.
- scikit-learn shortcut:
`PCA(n_components=0.95)` picks d automatically.
- Correlated behavioural features compress dramatically: ~ 200 wallet features often collapse to 20–40 components at 95%.



The knee of this curve is the honest answer to “how many features do we really have?”

PCA in scikit-learn

Compression, reconstruction and leakage discipline

```
from sklearn.decomposition import PCA
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

pca_pipe = Pipeline([
    ("scale", StandardScaler()),
    ("pca", PCA(n_components=0.95, random_state=42)),
])
Z_train = pca_pipe.fit_transform(X_train)    # fit on TRAIN only
Z_test  = pca_pipe.transform(X_test)         # never refit on test

pca = pca_pipe.named_steps["pca"]
print(pca.n_components_)                    # e.g. 31 of 200
X_rec = pca.inverse_transform(Z_train)       # reconstruction
```

- PCA is a **fitted transformer**: fitting on the full dataset before splitting is leakage (Week 2's rule).
- Per-instance **reconstruction error** $\|x - \hat{x}\|^2$ is itself an anomaly score.

PCA variants for scale and nonlinearity

Same idea, engineered for production

Variant	What it changes	When to use
Randomized PCA	stochastic SVD, $\mathcal{O}(m d^2)$ vs $\mathcal{O}(m n^2)$	large n , $d \ll n$ (sklearn auto-selects)
Incremental PCA	mini-batch fitting via <code>partial_fit</code>	data exceeds RAM; streaming pipelines
Kernel PCA	kernel trick in feature space: nonlinear components	curved manifolds; small-to-medium m
Truncated SVD	skips centring; works on sparse matrices	TF-IDF text, one-hot / sparse features

- Kernel PCA has no labels to tune against: select kernel/ γ by **downstream task performance** or lowest reconstruction error.
- *Looking ahead: autoencoders (Week 6) generalise this: learned nonlinear encoders with an explicit decoder.*

Method selection at a glance

The Week 5 toolbox

Method	Model class		Key knobs	Reach for it when...
K-Means	centroid, hard		k , <code>n_init</code>	fast segmentation; roughly convex groups
DBSCAN	density		ϵ , <code>min_samples</code>	unknown k ; odd shapes; want a noise label
GMM	probabilistic	mix-ture	k , covariance type	elliptical clusters; soft membership; densities
PCA	linear projection		d / variance kept	compression, decorrelation, speed-ups
Kernel PCA	nonlinear	projection	kernel, γ	curved structure, moderate m

Standard applied recipe: scale $\rightarrow$ PCA (95%) $\rightarrow$ cluster $\rightarrow$ profile in *original* units $\rightarrow$ visualise with PCA 2D.

Roadmap

Why Unsupervised Learning

Clustering with K-Means

Density-Based Clustering: DBSCAN

Gaussian Mixture Models

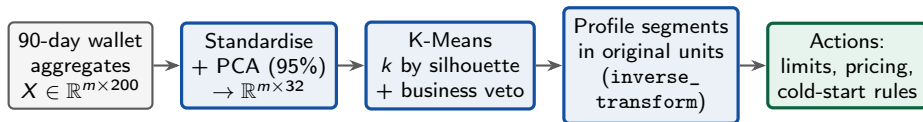
The Curse of Dimensionality & PCA

Applied Thread: Segmenting Thin-File Borrowers

Lab, Reading & Wrap-Up

Case study: segmentation for a mobile-money lender

From wallet logs to lending personas



Typical personas that emerge (illustrative):

Segment	Signature behaviour	Cash-flow rhythm	Lending implication
Market traders	many small P2P, float top-ups	daily, high velocity	short-cycle working capital
Gig workers	irregular inflows, app paybills	weekly, volatile	flexible repayment dates
Salaried users	one large monthly inflow	monthly, stable	standard instalments
Agri-cycle earners	post-harvest spikes	seasonal	bullet repayments

Governance: unsupervised $\neq$ unaccountable

Segments face regulators too

- **Interpretability:** a segment used for pricing must be explainable in plain terms, profile clusters in original feature units, not PCA coordinates.
- **Fairness:** clusters can silently reconstruct protected attributes (gender, region, ethnicity proxies such as location or paybill mix). Audit segment composition before attaching different prices or limits.
- **Data protection:** behavioural profiling falls under data-protection law (Kenya's Data Protection Act, 2019), lawful basis and purpose limitation apply to *derived* segments too.
- **Stability:** segments drift as the economy and product mix change. Version the clustering model, monitor segment populations, re-fit on a schedule, Week 13's drift material arriving early.

Looking ahead: in Weeks 11–12 the fitted scale $\rightarrow$ PCA $\rightarrow$ KMeans Pipeline ships as one artefact behind a FastAPI endpoint, no train/serve skew.

Roadmap

Why Unsupervised Learning

Clustering with K-Means

Density-Based Clustering: DBSCAN

Gaussian Mixture Models

The Curse of Dimensionality & PCA

Lab, Reading & Wrap-Up

Lab 5 (autonomous): customer segmentation & visualisation

Jupyter notebook: `lab04_unsupervised.ipynb`

Dataset: simulated mobile-money wallet aggregates ($m \approx 150,000$, $n = 187$, no labels), messy on purpose: mixed scales, correlated blocks, injected anomalies.

1. **Reduce:** standardise; PCA to 95% variance; plot the cumulative-variance curve and report d^* .
2. **Cluster:** K-Means for $k \in [2, 12]$, elbow + silhouette; fit DBSCAN and compare cluster structure and noise rate.
3. **Model densities:** GMM with BIC-selected k ; flag the bottom 0.5% by `score_samples` and inspect them.
4. **Visualise:** PCA 2D coloured by each method's labels; reconcile disagreements.
5. **Profile & present:** name each segment, one lending-relevant insight per segment, and one fairness concern.

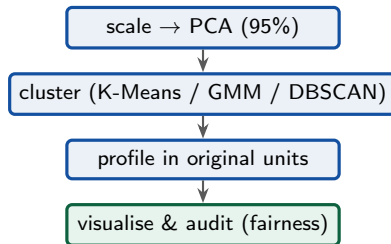
Reading & further material

- **Géron**, *Hands-On ML*, 2nd ed.: Ch. 8 (Dimensionality Reduction) and Ch. 9 (Unsupervised Learning Techniques), primary reading; the lab mirrors their code idioms.
- **Bishop**, *Deep Learning: Foundations and Concepts*: mixture models and EM chapters for the probabilistic depth behind today's GMM slides.
- Ester et al. (1996), the original DBSCAN paper; Campello et al. (2013) for HDBSCAN.

Week 5 summary

What must survive until the exam, and production

- Unsupervised learning finds **structure** in $p(x)$: clusters, densities, low-dim coordinates, anomalies.
- K-Means: fast, convex; k by elbow + silhouette; always scale.
- DBSCAN: density, shapes, noise. GMM: soft, elliptical; BIC selects k ; `score_samples` flags anomalies.
- PCA: top eigenvectors via SVD; keep 95% variance; fit on train only.
- PCA 2D/3D: visualisation, never features.



Next week: deep learning fundamentals, MLPs in TensorFlow, where representations are *learned* end-to-end.